
TD informatique

PANDÉMIE

Dans tout le problème, on peut utiliser une fonction traitée précédemment. On suppose que les bibliothèques `numpy` et `random` ont été importées par :

```
import numpy as np
import random as rd
```

L'étude de la propagation des épidémies joue un rôle important dans les politiques de santé publiques. Les modèles mathématiques ont permis de comprendre pourquoi il a été possible d'éradiquer la variole à la fin des années 1970 et pourquoi il est plus difficile d'éradiquer d'autres maladies comme la rougeole.

On s'intéresse dans ce problème une méthode de simulation numérique appelée *automates cellulaires*.

Dans ce qui suit, on appelle grille de taille $n \times n$ une liste de n listes de longueur n , où n est un entier strictement positif.

Pour mieux comprendre la dépendance spatiale de la contagion, il est possible de simuler la propagation d'une épidémie à l'aide d'une grille. Chaque case de la grille peut être dans un des quatre états suivants : saine, infectée, rétablie, décédée. On choisit de représenter ces quatre états par les entiers :

0 (Sain), 1 (Infecté), 2 (Rétabli) et 3 (Décédé)

L'état des cases d'une grille évolue au cours du temps selon des règles simples. On considère un modèle où l'état d'une case à l'instant $t + 1$ ne dépend que de son état à l'instant t et de l'état des ses huit cases voisines (cinq pour une case du bord et trois pour un coin). Les *règles de transition* sont les suivantes :

- une case décédée reste décédée
- une case infectée devient décédée avec une probabilité p_1 ou rétablie avec une probabilité $1 - p_1$
- une case rétablie reste rétablie
- une case saine devient infectée avec une probabilité p_2 si elle a au moins une case voisine infectée et reste saine sinon.

On initialise toutes les cases dans l'état sain, sauf une case choisie au hasard dans l'état infecté.

- 1) On a écrit la fonction `grille(n)` suivante :

```
def grille(n) :
    M=[ ]
    for i in range(n) :
        L=[ ]
        for j in range(n):
            L.append(0)
        M.append(L)
    return M
```

Décrire ce que retourne cette fonction.

- 2) Écrire une fonction `init(n)` qui construit une grille G de taille $n \times n$ ne contenant que des cases saines, puis choisit aléatoirement une des cases et la transforme en une case infectée.
On pourra utiliser la fonction `randrange(p)` de la bibliothèque `random` qui, pour un entier positif p , renvoie un entier choisi aléatoirement entre 0 et $p - 1$ inclus.
- 3) Écrire une fonction `compte(G)` qui a pour argument une grille G et renvoie la liste `[n0, n1, n2, n3]` formée des nombres de cases dans chacun des quatre états.

D'après les règles de transition, pour savoir si une case saine peut devenir infectée à l'instant suivant, il faut déterminer si elle est exposée à la maladie, c'est à dire si elle possède au moins une case infectée dans son voisinage. Pour cela on écrit la fonction `est_exposee(G, i, j)` suivante :

```

1  def est_exposee(G, i, j):
2      n = len(G)
3      if i == 0 and j == 0:
4          return (G[0][1]-1)*(G[1][1]-1)*(G[1][0]-1) == 0
5      elif i == 0 and j == n-1:
6          return (G[0][n-2]-1)*(G[1][n-2]-1)*(G[1][n-1]-1) == 0
7      elif i == n-1 and j == 0:
8          return (G[n-1][1]-1)*(G[n-2][1]-1)*(G[n-2][0]-1) == 0
9      elif i == n-1 and j == n-1:
10         return (G[n-1][n-2]-1)*(G[n-2][n-2]-1)*(G[n-2][n-1]-1) == 0
11     elif i == 0:
12         # a completer
13     elif i == n-1:
14         return (G[n-1][j-1]-1)*(G[n-2][j-1]-1)*(G[n-2][j]-1)*(G[n-2][j+1]-1)*(G[n-1][j+1]-1) == 0
15     elif j == 0:
16         return (G[i-1][0]-1)*(G[i-1][1]-1)*(G[i][1]-1)*(G[i+1][1]-1)*(G[i+1][0]-1) == 0
17     elif j == n-1:
18         return (G[i-1][n-1]-1)*(G[i-1][n-2]-1)*(G[i][n-2]-1)*(G[i+1][n-2]-1)*(G[i+1][n-1]-1) == 0
19     else:
20         # a completer

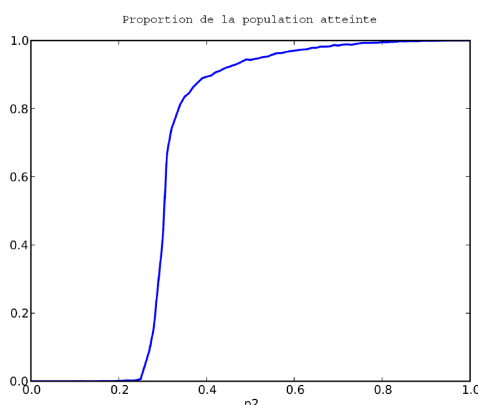
```

- 4) Quel est le type du résultat renvoyé par la fonction `est_exposee`?
- 5) Compléter les lignes 12 et 20 de la fonction `est_exposee` (NE PAS recopier tout le code)
- 6) Écrire une fonction suivant `(G, p1, p2)` qui fait évoluer toutes les cases de la grille `G` à l'aide des règles de transition et renvoie une nouvelle grille correspondant à l'instant suivant. Les arguments `p1` et `p2` sont les probabilités qui interviennent dans les règles de transition pour les cases infectées et les cases saines. On pourra utiliser la fonction `bernoulli(p)` qui simule une variable aléatoire de Bernoulli de paramètre p : `bernoulli(p)` renvoie 1 avec probabilité p et 0 avec probabilité $1-p$.

Avec les règles de transition du modèle utilisé, l'état de la grille évolue entre les instant t et $t+1$ tant qu'il existe au moins une case infectée.

- 7) Écrire une fonction `simulation(n, p1, p2)` qui réalise une simulation complète avec une grille de taille $n \times n$ pour les probabilités `p1` et `p2`, et renvoie la liste `[x0, x1, x2, x3]` des proportions de cases dans chacun des quatre états à la fin de la simulation (qui s'arrête lorsque la grille n'évolue plus).
- 8) Quelle est la valeur de la proportion des cases infectées `x1` à la fin d'une simulation? Quelle relation vérifient `x0`, `x1`, `x2` et `x3`? Comment obtenir avec ces valeurs la valeur `x_atteinte` de la population des cases qui ont été atteintes par la maladie pendant une simulation?

On fixe `p1` à 0,5 et on calcule la moyenne des résultats de plusieurs simulations pour différentes valeurs de `p2`. On obtient la courbe suivant donnant la proportion de la population qui a été atteinte par la maladie en fonction de `p2`.



- 9) On appelle seuil critique de pandémie la valeur de `p2` à partir de laquelle plus de la moitié de la population a été atteinte par la maladie à la fin de la simulation. On suppose que les valeurs de `p2` et `x_atteinte` utilisées pour tracer la figure ont été stockées dans deux listes de même longueur `Lp2` et `Lxa`. Écrire une fonction `seuil(Lp2, Lxa)` qui détermine par dichotomie un encadrement `[p2cim, p2cmax]` du seuil critique de pandémie avec la plus grande précision possible. On supposera que la liste `Lp2` croît de 0 à 1 et que la liste `Lxa` des valeurs correspondantes est croissante.
- 10) Pour étudier l'effet d'une campagne de vaccination, on immunise au hasard à l'instant initial une fraction `q` de la population. Écrire une fonction `init_vac(n, q)` qui construit une grille `G` ne contenant que des cases saines, et prend des cases au hasard qu'elle place en rétablie jusqu'à ce que la proportion de cases rétablies dépasse la proportion `q`.